MODULE-2

COMPUTER NETWORKS

Course Code BCS502

MODULE-2 Data Link Layer: Error Detection and Correction: Introduction, Block Coding, Cyclic Codes. Data link control: DLC Services: Framing, Flow Control, Error Control, Connectionless and Connection Oriented, Data link layer protocols, High Level Data Link Control. Media Access Control: Random Access, Controlled Access. Check Sum and Point to Point Protocol Textbook: Ch. 10.1-10.4, 11.1 -11.4, 12.1 - 12.2

2.1 INTRODUCTION

During data transmission, bits travel from one point to another through a communication channel. While traveling, these bits are exposed to various kinds of interference such as noise, distortion, and signal attenuation. Because of these disturbances, the received data may not always be identical to the transmitted data. This leads to errors in data communication. To ensure reliable communication, techniques for **error detection and error correction** are required.

2.1.1 Types of Errors

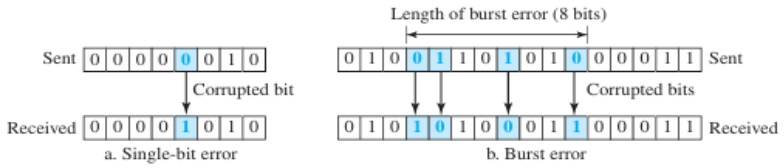
Whenever bits are transmitted over a communication medium, they may change unpredictably due to interference. Such interference can distort the signal and alter one or more bits in the data unit. Based on the number of bits affected, errors are classified into two types: **single-bit errors** and **burst errors**.

A **single-bit error** occurs when only one bit in a data unit, such as a byte, character, or packet, is changed from 0 to 1 or from 1 to 0. This type of error affects only a single position in the data.

A **burst error** occurs when two or more bits in a data unit are changed. In a burst error, the errors occur over a sequence of bits. Burst errors are more common than single-bit errors because the duration of noise is usually longer than the time required to transmit one bit. As a result, when noise affects the signal, it often corrupts multiple consecutive bits.

The number of bits affected in a burst error depends on both the **data rate** and the **duration of the noise**. For example, if data is transmitted at a rate of 1 kbps, a noise lasting 1/100 second can affect 10 bits. If the transmission rate is increased to 1 Mbps, the same noise duration can affect as many as 10,000 bits. Therefore, higher data rates are more susceptible to larger burst errors.

Figure 10.1 *Single-bit and burst error*



2.1.2 Redundancy

The fundamental concept behind error detection and correction is **redundancy**. Redundancy means adding extra bits to the original data before transmission. These additional bits do not carry new information, but they help the receiver determine whether the data has been corrupted during transmission.

The redundant bits are added by the **sender** and removed by the **receiver** after checking the data. By examining these extra bits, the receiver can detect the presence of errors and, in some cases, correct them. Without redundancy, it is impossible to detect or correct errors in received data.

2.1.3 Detection versus Correction

Error detection is simpler than **error correction**. In error detection, the receiver only checks whether an error has occurred or not. The result is a simple yes-or-no decision. The receiver does not need to know how many bits are corrupted or where the errors occurred. From the point of view of detection, a single-bit error and a burst error are treated in the same way.

Error correction, on the other hand, is more complex. In error correction, the receiver must determine not only that an error has occurred but also the **exact number of corrupted bits and their positions** in the data unit. The complexity of correction increases with the size of the data unit and the number of errors. For example, correcting one error in an 8-bit data unit requires checking 8 possible bit positions. Correcting two errors requires considering 28 possible combinations. If a large data unit contains many errors, locating and correcting them becomes extremely difficult.

2.1.4 Coding

Redundancy is introduced into data using **coding schemes**. In coding, the sender adds redundant bits to the data in such a way that a specific relationship is created between the data bits and the redundant bits. At the receiver, this relationship is checked to determine whether an error has occurred.

The effectiveness of a coding scheme depends on factors such as the **ratio of redundant bits to data bits** and the **strength or robustness** of the coding technique. Stronger coding schemes can detect and correct more errors but usually require more redundant bits.

Coding schemes are broadly classified into two types: **block coding** and **convolution coding**. In this book, the focus is mainly on **block coding**, while convolution coding is discussed less extensively.

2.2 BLOCK CODING

Block coding is a method used in error detection and correction in which the message is divided into fixed-size blocks. Each block contains **k bits**, known as a **dataword**. To enable error detection, the sender adds **r redundant bits** to each dataword. As a result, the total length of the transmitted block becomes **n = k + r bits**, which is called a **codeword**.

Each dataword is always encoded into the same codeword, which means block coding is a **one-to-one mapping** process. With k bits, it is possible to generate **2^k different datawords**, while with n bits, **2^n different codewords** are possible. Since n is greater than k, the number of possible codewords is larger than the number of datawords. Therefore, many codewords are **unused**. These unused codewords are known as **invalid or illegal codewords**.

The existence of invalid codewords is the key idea behind error detection. If the receiver receives a codeword that does not belong to the list of valid codewords, it can conclude that the data has been corrupted during transmission.

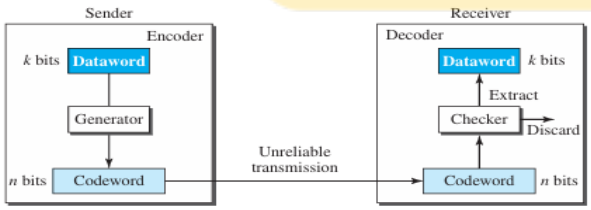
2.2.1 Error Detection Using Block Coding

Error detection using block coding is possible if two conditions are satisfied. First, the receiver must know the list of valid codewords. Second, the transmitted codeword must change into an invalid codeword due to errors during transmission.

At the sender side, datawords are converted into codewords using an **encoder** and a **generator** that follow predefined encoding rules. During transmission, the codeword may get corrupted due to noise. At the receiver side, the received codeword is checked. If it matches a valid codeword, it is accepted and the corresponding dataword is extracted. If it does not match any valid codeword, it is rejected. However, if a corrupted codeword accidentally matches another valid codeword, the error remains undetected.

Thus, an error-detecting code can detect only those errors for which it is designed.

Figure 10.2 *Process of error detection in block coding*



Example 10.1 Let us assume that k = 2 and n = 3. Table 10.1 shows the list of datawords and codewords. Later, we will see how to derive a codeword from a dataword.

Table 10.1 *A code for error detection in Example 10.1*

Dataword	*Codeword*	*Dataword*	*Codeword*
00	000	10	101
01	011	11	110

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.

2. The codeword is corrupted during transmission, and 111 is received (the leftmost bit is cor rupted). This is not a valid codeword and is discarded.

3. The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Hamming Distance

One of the most important concepts in block coding is **Hamming distance**. The Hamming distance between two binary words of the same length is defined as the **number of bit positions in which the two words differ**. It is represented as d(x, y).

Hamming distance is important because it tells us how many bits have been corrupted during transmission. For example, if the transmitted codeword is 00000 and the received codeword is 01101, then three bits are different. Therefore, the Hamming distance is 3, which means three errors occurred.

The Hamming distance can be calculated easily by applying the **XOR operation** on the two codewords and counting the number of 1s in the result.

Example 10.2 Let us find the Hamming distance between two pairs of words. 1. The Hamming distance d(000, 011) is 2 because (000 ⊕ 011) is 011 (two 1s). 2. The Hamming distance d(10101, 11110) is 3 because (10101 ⊕ 11110) is 01011 (three 1s).

Minimum Hamming Distance for Error Detection

The **minimum Hamming distance (d_{min})** of a block code is the smallest Hamming distance between any pair of valid codewords in the code. The value of d_{min} determines the error-detecting capability of the code.

To guarantee the detection of up to **s errors**, the minimum Hamming distance must be **s + 1**. This ensures that a corrupted codeword cannot become another valid codeword after s or fewer errors. Although some codes may detect more errors in special cases, only **s errors are guaranteed** to be detected.

Geometrically, this can be visualized by placing a transmitted codeword at the center of a circle with radius s. All codewords formed by up to s errors lie inside or on the circle, while all other valid codewords must lie outside the circle.

To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be dmin =S+1

Linear Block Codes

Most block codes used in practice are **linear block codes**. In a linear block code, the XOR (modulo-2 addition) of any two valid codewords results in another valid codeword. Linear block codes are widely used because they are easier to analyze and implement compared to nonlinear codes.

The minimum Hamming distance in a linear block code can be found easily. It is equal to the **number of 1s in the non-zero valid codeword that has the smallest number of 1s**.

Parity-Check Code

The **parity-check code** is the simplest and most commonly used linear block code. In this method, a k-bit dataword is converted into an (k + 1)-bit codeword by adding one extra bit called the **parity bit**. The parity bit is chosen such that the total number of 1s in the codeword becomes **even** (even parity).

The parity bit is calculated using modulo-2 addition of all data bits. If the number of 1s in the dataword is odd, the parity bit is set to 1; if it is even, the parity bit is set to 0.

The minimum Hamming distance of a parity-check code is **2**, which means it can detect **any single-bit error**. It can also detect any **odd number of errors**, but it cannot detect even numbers of errors because such errors cancel each other.

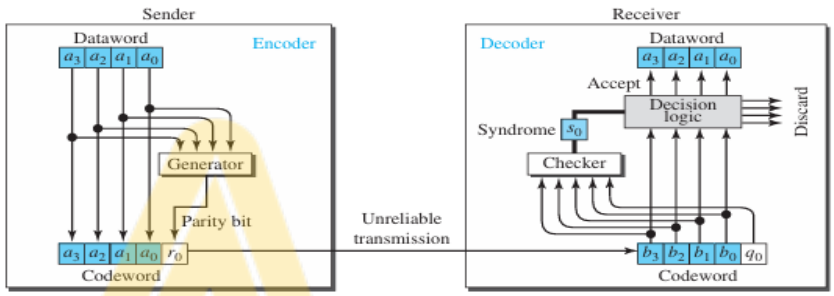
At the receiver, all bits of the received codeword are added modulo-2 to form a single bit called the **syndrome**. If the syndrome is 0, the codeword is accepted; if it is 1, the codeword is discarded.

Table 10.2 *Simple parity-check code C(5, 4)*

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	**00000**	1000	**10001**
0001	**00011**	1001	**10010**
0010	**00101**	1010	**10100**
0011	**00110**	1011	**10111**
0100	**01001**	1100	**11000**
0101	**01010**	1101	**11011**
0110	**01100**	1110	**11101**
0111	**01111**	1111	**11110**

Figure 10.4 shows a possible structure of an encoder (at the sender) and a decoder (at the receiver).

Figure 10.4 *Encoder and decoder for simple parity-check code*



Encoder Operation (Sender Side)

- The sender starts with a **4-bit dataword**: a0,a1,a2,a3a_0, a_1, a_2, a_3a0,a1,a2,a3
- A **parity bit r0r_0r0** is generated using **modulo-2 addition (XOR)** of all 4 data bits:

$$r_0 = a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

Modulo-2 addition rules:

- If the number of 1s in the dataword is:
  - **Even** → r0=0r_0 = 0r0=0
  - **Odd** → r0=1r_0 = 1r0=1
- The parity bit ensures that the **total number of 1s in the final 5-bit codeword is even**.

- The transmitted **5-bit codeword** is:

(a3,a2,a1,a0,r0)(a_3, a_2, a_1, a_0, r_0)(a3,a2,a1,a0,r0)

2. Transmission

- The codeword is sent through the communication channel.
- During transmission, **errors may occur**, changing one or more bits.

3. Checker Operation (Receiver Side)

- The receiver gets a **5-bit received word**:

(b3,b2,b1,b0,q0)(b_3, b_2, b_1, b_0, q_0)(b3,b2,b1,b0,q0)

- The receiver computes the **syndrome s0s_0s0** by adding **all 5 bits** using modulo-2 arithmetic:

$$s_0 = b_3 \oplus b_2 \oplus b_1 \oplus b_0 \oplus q_0$$

4. Syndrome Interpretation

- **Syndrome s0=0s_0 = 0s0=0**
  - Total number of 1s is even
  - **No detectable error**
  - Data bits are accepted as the correct dataword
- **Syndrome s0=1s_0 = 1s0=1**
  - Total number of 1s is odd
  - **Error detected**
  - Dataword is **discarded**

2.3 CYCLIC CODES

Cyclic codes are a special type of linear block codes used mainly for error detection in computer networks. A cyclic code has an additional important property: if a valid codeword is cyclically shifted (rotated), the result is also a valid codeword.
For example, if 1011000 is a codeword, then its cyclic left shift 0110001 is also a valid codeword. This cyclic property makes these codes very powerful and easy to implement.

Cyclic codes are widely used in practice because they provide strong error detection, especially for burst errors, and they can be implemented efficiently using hardware.

b1 = a0 b2 =a1 b3 =a2 b4 = a3 b5 = a4 b6 = a5

In the rightmost equation, the last bit of the first word is wrapped around and becomes the first bit of the second word.

2.3.1 Cyclic Redundancy Check (CRC)

The most commonly used cyclic code is the **Cyclic Redundancy Check (CRC)**. CRC is widely used in **LANs, WANs, Ethernet, HDLC, ATM, and many communication protocols**.

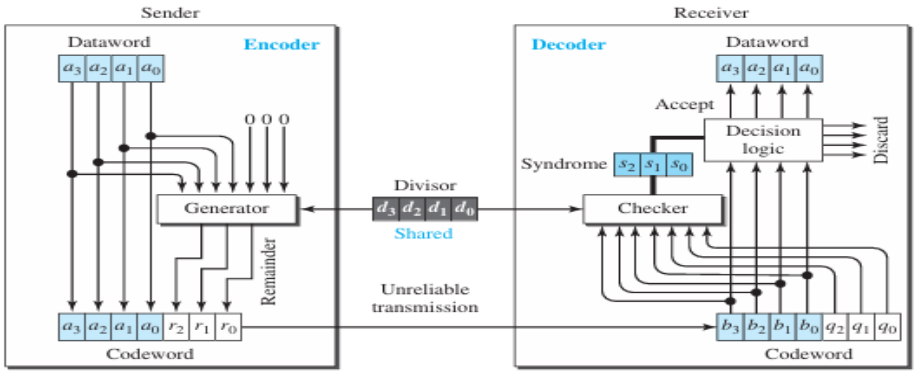
In CRC, a dataword of **k bits** is converted into a codeword of **n bits** by adding **r = n − k check bits**. These check bits are generated using a predefined binary pattern called the **divisor** or **generator**.

Table 10.3 *A CRC code with C(7, 4)*

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

Figure 10.5 shows one possible design for the encoder and decoder.

Figure 10.5 *CRC encoder and decoder*



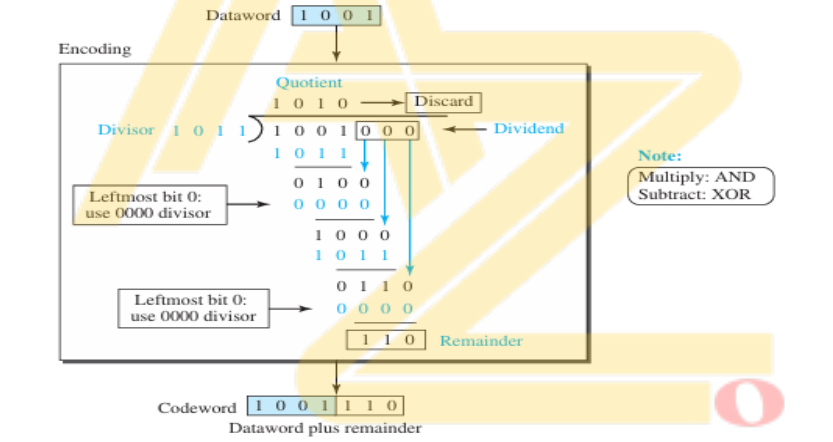
CRC Encoding Process

At the sender side, the following steps are performed:

1. The dataword is first **augmented** by adding **n − k zeros** to its right.
2. The augmented dataword is then divided by a predefined **divisor** using **modulo-2 division**.
3. The **quotient is discarded**, and the **remainder** obtained from the division becomes the **check bits**.
4. These remainder bits are appended to the original dataword to form the **codeword**.

Modulo-2 division is similar to normal binary division, except that **subtraction and addition are performed using XOR operations**.

Figure 10.6 *Division in CRC encoder*



CRC Decoding Process

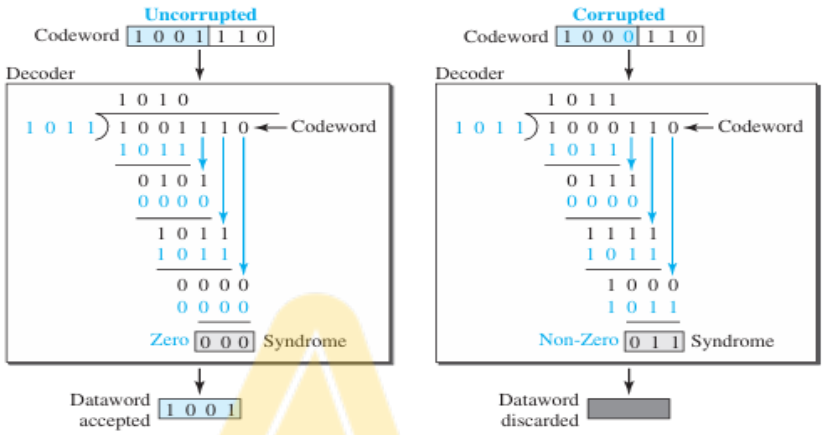
At the receiver side:

1. The received codeword (which may be corrupted) is divided by the same divisor.
2. The remainder of this division is called the **syndrome**.
3. If the syndrome is **all zeros**, the data is accepted as error-free.

4. If the syndrome is **non-zero**, an error is detected and the dataword is discarded.

CRC provides **very high reliability**, although it does not correct errors—it only detects them.

Figure 10.7 *Division in the CRC decoder for two cases*



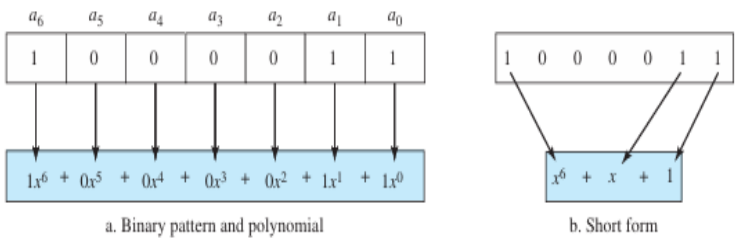
2.3.2 Polynomial Representation

To simplify analysis, cyclic codes are represented using **polynomials with binary coefficients (0 or 1)**.

- Each bit position corresponds to a power of **x**.
- A binary pattern such as 1011 is represented as the polynomial $\mathbf{x^3 + x + 1}$.
- The **degree of the polynomial** is one less than the number of bits.

Polynomial representation makes **division, multiplication, and error analysis easier** and clearer than working directly with bits.

Figure 10.8 *A polynomial to represent a binary word*



Polynomial Operations

x23 + x3 + 1. Here the bit pattern is 24 bits in length (three 1s and twenty-one 0s) while the polynomial is just three terms.

Degree of a Polynomial

The degree of a polynomial is the highest power in the polynomial. For example, the degree of the polynomial x6 + x + 1 is 6. Note that the degree of a polynomial is 1 less than the number of bits in the pattern. The bit pattern in this case has 7 bits.

Adding and Subtracting Polynomials

Adding and subtracting polynomials in mathematics are done by adding or subtracting the coefficients of terms with the same power. In our case, the coefficients are only 0 and 1, and adding is in modulo-2. This has two consequences. First, addition and sub traction are the same. Second, adding or subtracting is done by combining terms and deleting pairs of identical terms. For example, adding x5 + x4 + x2 and x6 + x4 + x2 gives just x6 + x5. The terms x4 and x2 are deleted. However, note that if we add, for example, three polynomials and we get x2 three times, we delete a pair of them and keep the third

Multiplying or Dividing Terms

In this arithmetic, multiplying a term by another term is very simple; we just add the powers. For example, x3 × x4 is x7. For dividing, we just subtract the power of the sec ond term from the power of the first. For example, x5/x2 is x3.

Multiplying Two Polynomials

Multiplying a polynomial by another is done term by term. Each term of the first polyno mial must be multiplied by all terms of the second. The result, of course, is then simplified, and pairs of equal terms are deleted. The following is an example: (x5 + x3 + x2 + x)(x2 + x + 1) = x7 + x6 + x5 + x5 + x4 + x3 + x4 + x3 + x2 + x3 + x2 + x = x7 + x6 + x3 + x

Dividing One Polynomial by Another

Division of polynomials is conceptually the same as the binary division we discussed for an encoder. We divide the first term of the dividend by the first term of the divisor to get the first term of the quotient. We multiply the term in the quotient by the divisor and subtract the result from the dividend. We repeat the process until the dividend degree is less than the divisor degree. We will show an example of division later in this chapter.

Shifting

A binary pattern is often shifted a number of bits to the right or left. Shifting to the left means adding extra 0s as rightmost bits; shifting to the right means deleting some right most bits. Shifting to the left is accomplished by multiplying each term of the polynomial by xm, where m is the number of shifted bits; shifting to the right is accomplished by dividing each term of the polynomial by xm. The following shows shifting to the left and to the right. Note that we do not have negative powers in the polynomial representation.

Shifting left 3 bits: 10011 becomes 10011000	x^4+x+1 becomes $x^7+x^4+x^3$
Shifting right 3 bits: 10011 becomes 10	x^4+x+1 becomes x

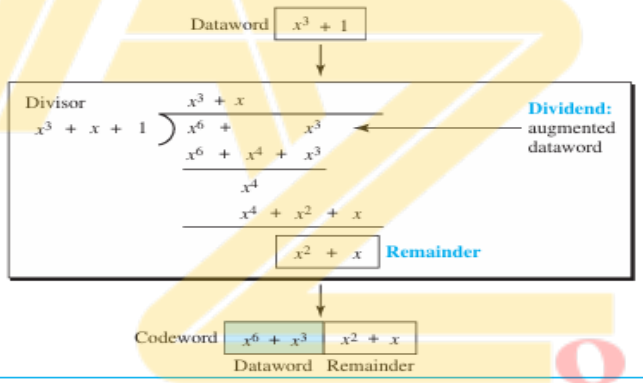
When we augmented the dataword in the encoder of Figure 10.6, we actually shifted the bits to the left. Also note that when we concatenate two bit patterns, we shift the first polynomial to the left and then add the second polynomial.

2.3.3 CRC Encoder Using Polynomials

In polynomial form:

- Dataword is represented as **d(x)**.
- Generator is represented as **g(x)**.
- Codeword is represented as **c(x)**.

Figure 10.9 *CRC division using polynomials*



- It can be seen that the polynomial representation can easily simplify the operation of division in this case, because the two steps involving all-0s divisors are not needed here. (Of course, one could argue that the all-0s divisor step can also be eliminated in binary division.) In a polynomial representation, the divisor is normally referred to as the generator polynomial t(x). The divisor in a cyclic code is normally called the generator polynomial or simply the generator.

The augmented dataword is obtained by multiplying **d(x)** by $\mathbf{x^r}$. After dividing this by **g(x)**, the remainder is added to form the final codeword.

The divisor polynomial is usually called the **generator polynomial**.

2.3.4 Cyclic Code Analysis

If the received codeword is **c(x) + e(x)**, where **e(x)** represents the error, then:

- The receiver divides the received codeword by **g(x)**.
- If the remainder (syndrome) is **non-zero**, an error is detected.
- If the remainder is **zero**, either no error occurred or the error went undetected.

Cyclic Codes – Polynomial Analysis

In cyclic codes, **binary sequences are represented as polynomials** with coefficients 0 or 1. Error detection capability is analyzed using **polynomial division**.

Definitions

- **Dataword:** d(x)d(x)d(x)
- **Codeword:** c(x)c(x)c(x)
- **Generator polynomial:** g(x)g(x)g(x)
- **Error polynomial:** e(x)e(x)e(x)
- **Syndrome:** s(x)s(x)s(x)

Relationship Between Codeword, Error, and Received Word

When a codeword is transmitted, it may get corrupted during transmission.

$$\text{Received codeword} = c(x) + e(x)$$

This means the received polynomial is the **sum (modulo-2)** of the transmitted codeword and the error pattern.

Syndrome Calculation

At the receiver, the **received codeword is divided by the generator polynomial g(x)**

$$s(x) = \frac{c(x) + e(x)}{g(x)}$$

Using polynomial division rules:

$$\frac{c(x) + e(x)}{g(x)} = \frac{c(x)}{g(x)} + \frac{e(x)}{g(x)}$$

- Since $c(x)$ is a valid codeword, it is **exactly divisible by** $g(x)$.
- Therefore, the remainder of $\frac{c(x)}{g(x)} = 0$.

Hence,

$$s(x) = \text{remainder of } \frac{e(x)}{g(x)}$$

So, **the syndrome depends only on the error polynomial**.

Interpretation of Syndrome

Case 1: s(x)≠0

- The error polynomial **is not divisible by g(x)**
- **One or more bits are corrupted**.
- The error is **detected**.

Case 2: s(x)=0

This can happen in two situations:

1. **No error occurred**
   - e(x)=0
2. **Error occurred but not detected**
   - e(x) is **divisible by g(x)**

Errors whose polynomial representations are divisible by the generator polynomial **cannot be detected**.

Errors that are **divisible by the generator polynomial are not detected**.

Error Detection Capabilities of CRC

Single-Bit Errors

If the generator polynomial has:

- More than one term, and
- A non-zero constant term,

then **all single-bit errors are guaranteed to be detected**.

Two Isolated Bit Errors

CRC can detect two isolated bit errors if the generator polynomial **does not divide $x^t + 1$**, where *t* is the distance between the two error bits.

Odd Number of Errors

If the generator polynomial contains the factor **(x + 1)**, then **all odd-numbered errors are detected**.

Burst Errors

CRC is especially strong at detecting burst errors:

- All burst errors of length ≤ **r** are detected.
- Burst errors of length **r + 1** are detected with very high probability.
- Longer burst errors are detected with probability close to 1.

Properties of a Good Generator Polynomial

A good CRC generator polynomial should:

1. Have at least **two terms**.

2. Have a **constant term of 1**.
3. Not divide $\mathbf{x^t + 1}$ for valid values of t.
4. Contain the factor **(x + 1)**.

Standard CRC Polynomials

Some commonly used CRC polynomials are:

- **CRC-8**
- **CRC-10**
- **CRC-16**
- **CRC-32**

CRC-32 is widely used in Ethernet and modern networks because of its excellent error detection capability.

2.3.5 Advantages of Cyclic Codes

Cyclic codes:

- Detect single-bit, double-bit, odd-numbered, and burst errors efficiently.
- Are easy to implement in both **hardware and software**.
- Are very fast when implemented using **shift registers and XOR gates**.
- Are widely adopted in real-world communication systems.

2.3.6 Other Cyclic Codes

More advanced cyclic codes, such as **Reed–Solomon codes**, are used for both **error detection and correction** in CDs, DVDs, QR codes, and storage systems. These codes are based on advanced mathematics and are beyond the scope of this discussion.

2.3.7 Hardware Implementation

CRC encoders and decoders can be implemented using:

- **Shift registers**
- **XOR gates**

This hardware-based design is:

- Low-cost
- Fast
- Efficient for real-time communication

Because of these advantages, CRC is preferred in high-speed networks.

2.4 CHECKSUM

A **checksum** is an **error-detecting technique** that can be applied to a message of **any length**. In computer networks, checksum is mainly used at the **network layer and transport layer** (for example, IP, TCP, and UDP), rather than the data-link layer. It helps the receiver determine whether the data has been corrupted during transmission.

Basic Working of Checksum

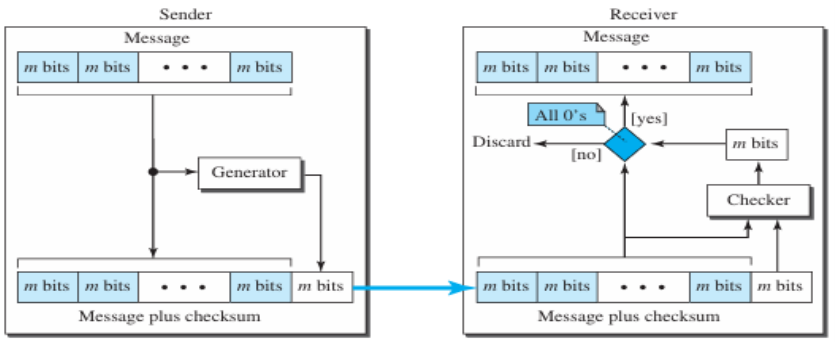
At the **sender side**, the message is divided into fixed-size units of **m bits**. Using these units, an extra **m-bit value called the checksum** is calculated and sent along with the message.

At the **receiver side**, the same checksum calculation is performed on the received data and the received checksum.

- If the final result is **all zeros**, the message is assumed to be error-free and is accepted.
- If the result is **not zero**, an error is detected and the message is discarded.

The checksum field does not always need to be placed at the end of the message; it may appear anywhere within the message.

Figure 10.15 *Checksum*



2.4.1 Concept of Checksum

The idea of checksum is based on **simple addition**. The sender adds all parts of the message and sends the result (or its complement) along with the message. The receiver performs the same operation to verify correctness.

Simple Example

Suppose the sender wants to transmit five numbers. Along with these numbers, the sender also sends their **sum**.
At the receiver side, the numbers are added again and compared with the transmitted sum.

- If both values match, the data is accepted.
- If they do not match, an error is assumed.

However, this basic method has limitations when the sum exceeds the fixed word size.

One's Complement Arithmetic

To solve the problem of limited word size, **one's complement arithmetic** is used.

In one's complement arithmetic:

- Numbers are represented using a fixed number of bits.
- If addition produces extra carry bits, the carry is **wrapped around and added back** to the result.
- The complement of a number is obtained by **changing all 0s to 1s and all 1s to 0s**.

This method allows checksum calculation without increasing the number of bits.

Checksum Using One's Complement

Instead of sending the sum itself, the sender sends the **one's complement of the sum**, which is called the **checksum**.

At the receiver:

1. All data words and the checksum are added using one's complement arithmetic.
2. If the result is **all 1s**, it is complemented again to get **0**.
3. A result of **0** indicates no detected error.

This approach simplifies error checking at the receiver.

Internet Checksum

The Internet commonly uses a **16-bit checksum**. This checksum is used in protocols such as **IP, TCP, and UDP**.

Sender Side Steps

1. Divide the message into **16-bit words**.
2. Initialize the checksum field to **0**.
3. Add all 16-bit words using **one's complement addition**.
4. Complement the final sum to get the checksum.
5. Send the checksum along with the data.

Receiver Side Steps

1. Receive the message and checksum.
2. Divide the message into **16-bit words**.
3. Add all words (including checksum) using one's complement addition.

4. Complement the result.

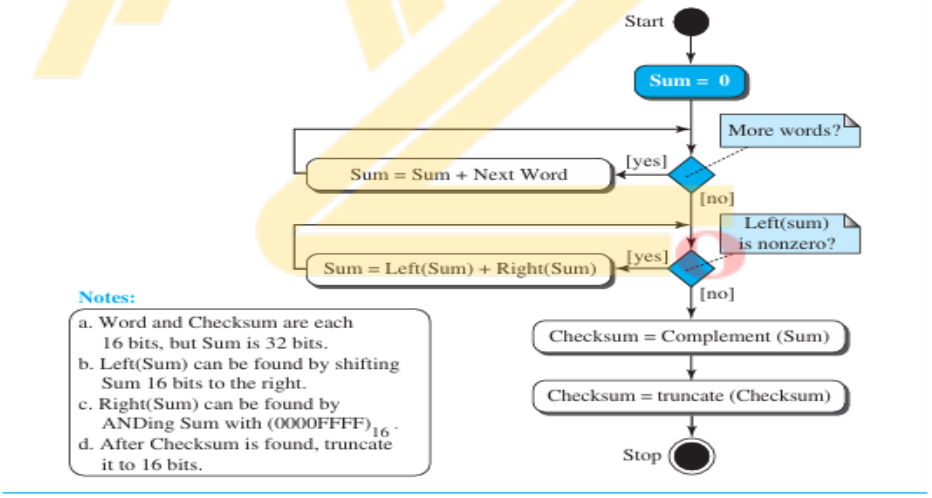
5. If the result is **0**, accept the message; otherwise, reject it.

Table 10.5 *Procedure to calculate the traditional checksum*

Sender	*Receiver*
1. The message is divided into 16-bit words.	1. The message and the checksum are received.
2. The value of the checksum word is initially set to zero.	2. The message is divided into 16-bit words.
3. All words including the checksum are added using one's complement addition.	3. All words are added using one's complement addition.
4. The sum is complemented and becomes the checksum.	4. The sum is complemented and becomes the new checksum.
5. The checksum is sent with the data.	5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

Algorithm We can use the flow diagram of Figure 10.17 to show the algorithm for calculation of the checksum. A program in any language can easily be written based on the algorithm. Note that the first loop just calculates the sum of the data units in two's complement; the second loop wraps the extra bits created from the two's complement calculation to simulate the calculations in one's complement. This is needed because almost all com puters today do calculation in two's complement.

Figure 10.17 *Algorithm to calculate a traditional checksum*



Performance of Checksum

The checksum technique uses **only a small number of bits** to detect errors in a large message. However, it is **not as strong as CRC**.

Some limitations of checksum are:

- Certain errors may cancel each other out and remain undetected.
- Reordering (transposition) of data words cannot be detected.
- If one word increases and another decreases by the same amount, the checksum may remain unchanged.

Because of these limitations, modern protocols increasingly prefer **CRC** over checksum.

2.4.2 Other Approaches to Checksum

To overcome weaknesses of the traditional checksum, **weighted checksum methods** have been introduced.

Fletcher Checksum

The **Fletcher checksum** assigns weights to data based on their position, which improves error detection.

- **8-bit Fletcher checksum**
  - Operates on 8-bit data units
  - Produces a 16-bit checksum
  - Uses modulo 256 arithmetic
- **16-bit Fletcher checksum**
  - Operates on 16-bit data units
  - Produces a 32-bit checksum
  - Uses modulo 65,536 arithmetic

Fletcher checksum uses two accumulators:

- One for the running sum
- One for weighted accumulation

Adler Checksum

The **Adler checksum** is a 32-bit checksum and is an improvement over Fletcher checksum.

Key features:

- Operates on **bytes (8 bits)** instead of 16-bit words
- Uses a **prime modulus (65,521)** for better error detection
- Initializes one accumulator to **1 instead of 0**

Adler checksum provides better performance for certain data patterns and is used in compression algorithms like **zlib**.

Checksum is a **simple and fast error-detection method** used in network protocols. While it is easy to compute and suitable for software implementation, it is **weaker than CRC** in detecting complex errors. As a result, modern communication systems increasingly rely on **CRC**, while checksum remains useful for lightweight error detection.

2.5 DLC SERVICES

Data Link Control (DLC) is responsible for **node-to-node communication**, that is, communication between two directly connected devices. This communication can occur over either a **dedicated link** or a **broadcast link**. The main functions of DLC are **framing**, **flow control**, and **error control**. These functions ensure that data is transmitted reliably and efficiently over the physical medium.

2.5.1 Framing

The **physical layer** transfers raw bits in the form of signals from the sender to the receiver and provides **bit synchronization**. However, it does not understand how these bits are organized.

The **data-link layer** groups these bits into meaningful units called **frames**. Framing helps the receiver identify **where one message starts and where it ends**.

An easy real-life example of framing is the **postal system**. Placing a letter inside an envelope separates it from other letters. Similarly, a frame separates one data message from another. The frame also contains **source and destination addresses**, which help in identifying the sender and receiver.

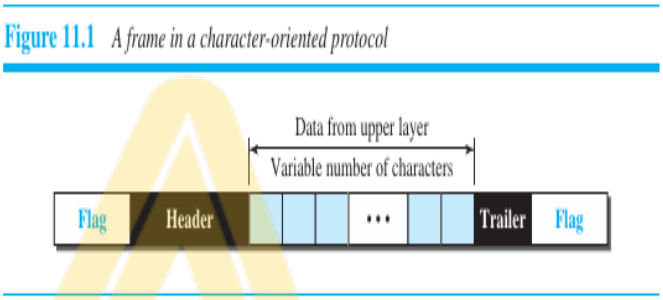
Instead of sending the entire message as one large frame, the message is divided into **smaller frames**. This improves efficiency because if an error occurs, **only the affected frame needs to be retransmitted**, not the whole message.

Frame Size

Frames can be of **fixed size** or **variable size**.

- In **fixed-size framing**, all frames have the same size, so frame boundaries are automatically known.
  Example: **ATM networks**, which use fixed-size cells.

- In **variable-size framing**, frames have different sizes, so **special techniques are needed** to mark the start and end of each frame.
  This method is commonly used in **LANs**.



There are two main approaches for variable-size framing:

1. Character-oriented framing
2. Bit-oriented framing

Character-Oriented Framing

In **character-oriented framing**, data is treated as a sequence of **8-bit characters** (such as ASCII). The frame begins and ends with a special **flag byte** that marks frame boundaries.

Problem:

If the same flag character appears inside the data, the receiver may mistakenly treat it as the end of the frame.

Solution: Byte Stuffing

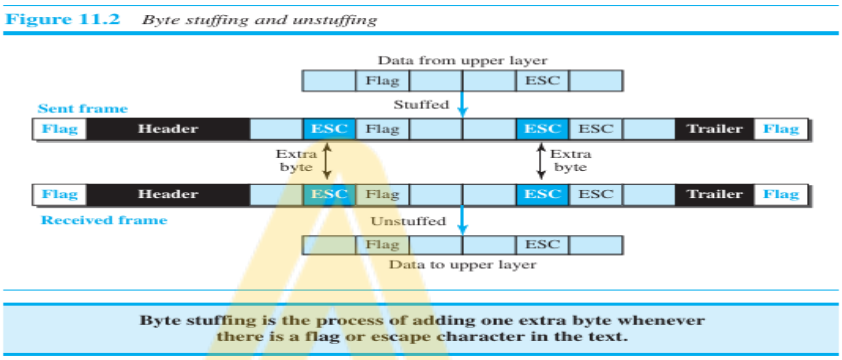
In **byte stuffing**, an extra special byte called an **escape character (ESC)** is inserted before any data byte that matches the flag or ESC pattern.

At the receiver:

- The ESC character is removed.
- The following byte is treated as **data**, not as a flag.

If the data itself contains the ESC character, another ESC is added to distinguish it from control information.

Due to modern data formats like **Unicode (16-bit and 32-bit characters)**, character-oriented framing has become less popular.



Bit-Oriented Framing

In **bit-oriented framing**, the data is treated as a **stream of bits**, not characters. This method supports all types of data such as text, audio, video, and images.

A special **8-bit flag pattern (01111110)** is used to mark the beginning and end of a frame.

Problem:

The same bit pattern may appear inside the data.

Solution: Bit Stuffing

In **bit stuffing**, whenever **five consecutive 1s** appear in the data, the sender inserts an extra **0** after them.

At the receiver:

- The extra 0 is removed.
- The original data is restored.

This ensures that the flag pattern never appears inside the data unintentionally.

Figure 11.1 *A frame in a character-oriented protocol*

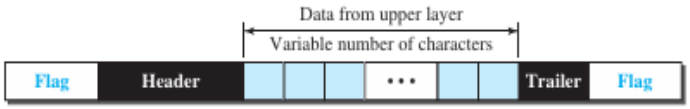
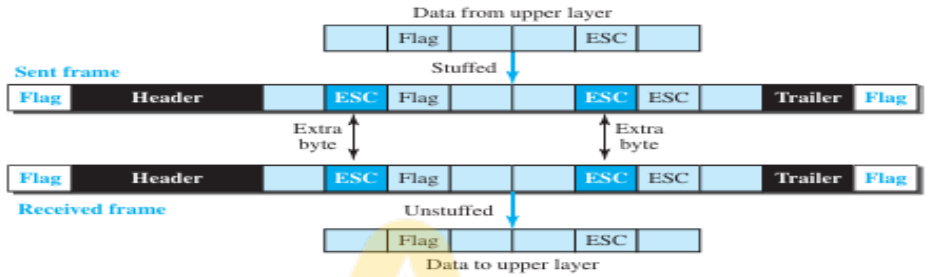


Figure 11.2 *Byte stuffing and unstuffing*



Byte stuffing is the process of adding one extra byte whenever there is a flag or escape character in the text.

\

2.5.2 Flow and Error Control

The data-link layer also performs **flow control** and **error control** to ensure reliable communication.

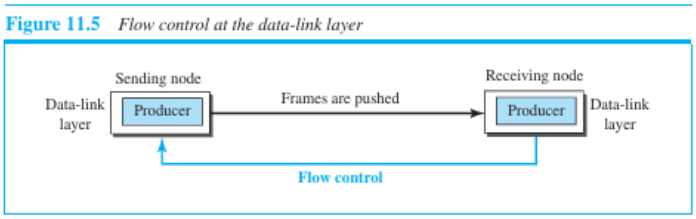
Flow Control

Flow control ensures that the sender does not transmit frames faster than the receiver can process them.

If the sender sends frames too quickly:

- The receiver's buffer may overflow.
- Frames may be lost.

To prevent this, the receiver sends **feedback signals** to the sender to slow down or stop transmission when needed.



Buffers

Both the sender and receiver use **buffers**, which are temporary memory areas to store frames. When the receiver's buffer becomes full, it informs the sender to pause transmission.

Error Control

Because the physical layer is not fully reliable, errors can occur during transmission. The data-link layer uses **CRC (Cyclic Redundancy Check)** to detect errors.

There are two common approaches:

1. If a corrupted frame is detected, it is **discarded silently** (used in Ethernet).
2. If a frame is received correctly, the receiver sends an **acknowledgment (ACK)** to the sender.

Combination of Flow and Error Control

Flow control and error control often work together.
An **ACK frame** can confirm:

- Correct reception of data (error control)
- Readiness to receive more data (flow control)

If no ACK is received, the sender assumes an error occurred.

2.5.3 Connectionless and Connection-Oriented DLC

Data-link protocols can be **connectionless** or **connection-oriented**.

Connectionless Protocol

In a **connectionless protocol**:

- Each frame is sent independently.
- Frames are not numbered.
- There is no setup or teardown phase.

Most **LAN protocols** (such as Ethernet) use connectionless communication.

Connection-Oriented Protocol

In a **connection-oriented protocol**:

1. A logical connection is established (setup phase).
2. Frames are sent in sequence (transfer phase).
3. The connection is terminated (teardown phase).

Frames are **numbered**, and the receiver ensures they are delivered **in order** to the network layer.

Connection-oriented DLC is commonly used in:

- Point-to-point links
- Some wireless networks
- Some WAN technologies

2.6 DATA-LINK LAYER PROTOCOLS

The **data-link layer** uses specific protocols to handle **flow control** and **error control** between two directly connected nodes. Traditionally, **four protocols** were defined for this purpose:
Simple, **Stop-and-Wait**, **Go-Back-N**, and **Selective Repeat**.

Among these, **Simple** and **Stop-and-Wait** protocols are still relevant at the data-link layer, especially for understanding **wired and wireless LANs**. The other two protocols are now mainly discussed at the **transport layer** and are explained in detail in Chapter 23.

Finite State Machine (FSM)

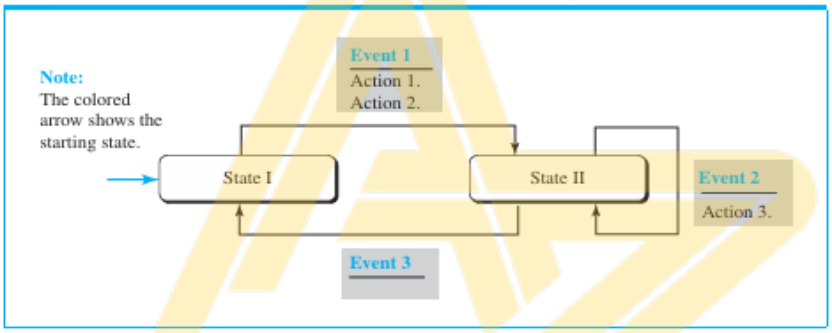
The behavior of data-link layer protocols is often explained using a **Finite State Machine (FSM)**.

An FSM is a model that consists of:

- A **finite number of states**
- **Events** that cause transitions
- **Actions** performed during transitions

The machine always starts in an **initial state** and moves to another state when an event occurs. Each event may trigger one or more actions and lead to a new state or remain in the same state. FSMs help clearly represent how protocols behave during communication.

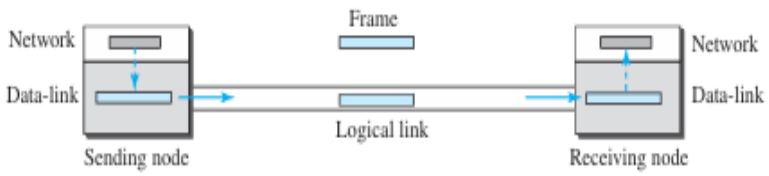
Figure 11.6 *Connectionless and connection-oriented service represented as FSMs*



2.6.1 Simple Protocol

The **Simple Protocol** is the most basic data-link layer protocol.
It provides **neither flow control nor error control**.

Figure 11.7 *Simple protocol*

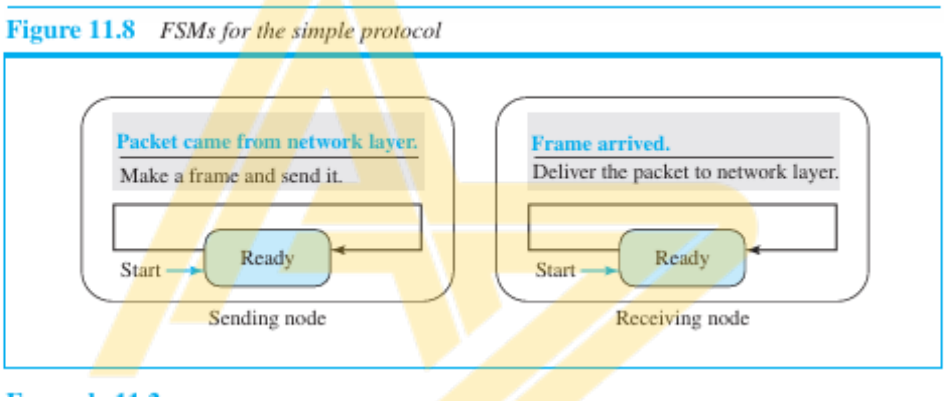


Assumptions:

- The receiver can always process incoming frames immediately.
- No frames are lost or corrupted.
- The receiver never becomes overloaded.

Working:

- The sender receives a packet from its network layer.
- It encapsulates the packet into a frame and sends it.
- The receiver extracts the packet from the frame and delivers it to its network layer.



FSM Behavior:

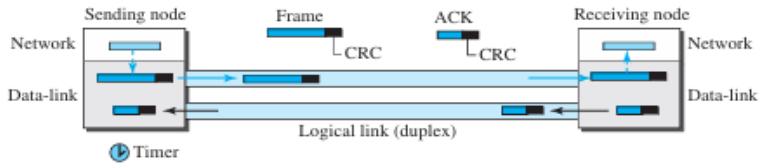
- Both sender and receiver have **only one state**, called the **ready state**.
- The sender waits for a packet from the network layer.
- The receiver waits for a frame from the sender.

This protocol is **very simple**, but **not practical** in real networks because it ignores errors and receiver limitations.

2.6.2 Stop-and-Wait Protocol

The **Stop-and-Wait Protocol** improves the simple protocol by providing **both flow control and error control**.

Figure 11.10 *Stop-and-Wait protocol*



Basic Idea:

- The sender sends **one frame at a time**.
- After sending a frame, the sender **waits for an acknowledgment (ACK)**.
- Only after receiving the ACK does the sender send the next frame.

Error Detection:

- Each frame contains a **CRC**.
- If a frame arrives corrupted, it is **discarded silently**.

Timer Mechanism:

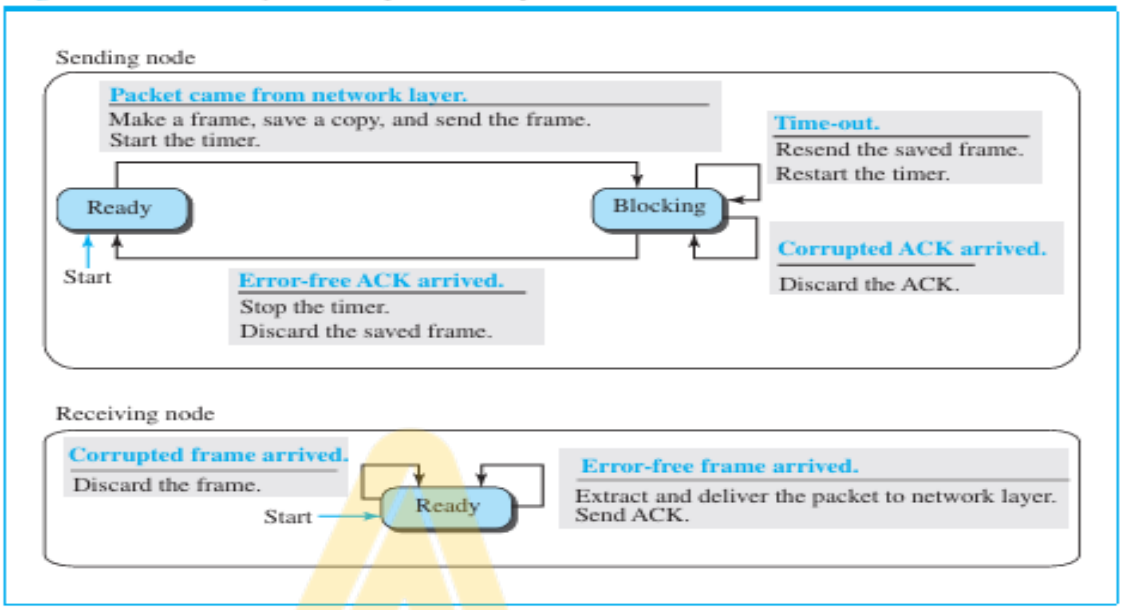
- After sending a frame, the sender **starts a timer**.
- If an ACK arrives before the timer expires, the sender proceeds.
- If the timer expires, the sender **resends the same frame**, assuming it was lost or corrupted.
- The sender keeps a copy of the frame until its ACK arrives.

At any moment, **only one frame and one ACK** can be in transit.

FSMs

Figure 11.11 shows the FSMs for our primitive Stop-and-Wait protocol.

Figure 11.11 *FSM for the Stop-and-Wait protocol*



Sender States in Stop-and-Wait Protocol

The sender has **two states**:

1. Ready State

- The sender waits for a packet from the network layer.
- When a packet arrives, it:
  - Creates a frame
  - Saves a copy
  - Sends the frame
  - Starts the timer
- Then it moves to the **blocking state**.

2. Blocking State

In this state, the sender waits for an ACK. Three events may occur:

- **Timeout** → resend the saved frame and restart the timer.
- **Corrupted ACK** → discard the ACK.
- **Error-free ACK** → stop the timer, discard the saved frame, and return to the ready state.

Receiver Behavior in Stop-and-Wait Protocol

The receiver always remains in the **ready state**.

- If an **error-free frame** arrives:
    - The packet is delivered to the network layer.
    - An ACK is sent back to the sender.
- If a **corrupted frame** arrives:
    - The frame is discarded.

Problem of Duplicate Frames

A major issue with the primitive Stop-and-Wait protocol is **duplicate packets**.

If:

- A frame is correctly received, but
- Its ACK is lost,

then the sender retransmits the frame.
As a result, the receiver may deliver the **same packet twice** to its network layer.

This problem must be corrected.

Sequence and Acknowledgment Numbers

To prevent duplicate packets, **sequence numbers** and **acknowledgment numbers** are added.

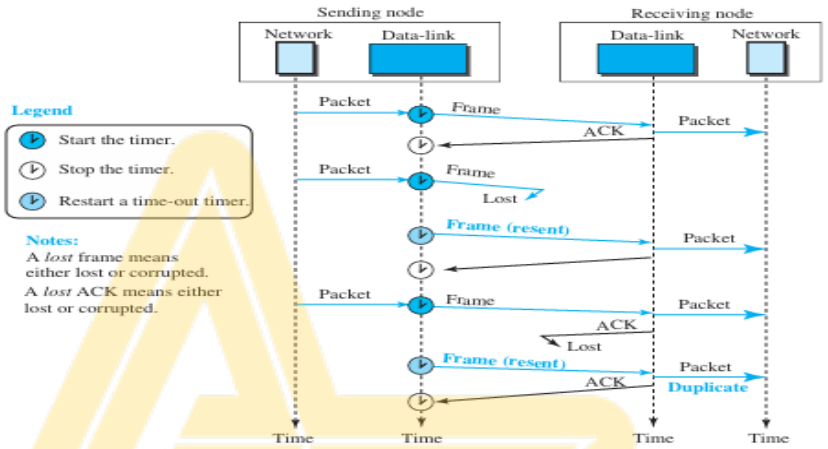
- Sequence numbers alternate between **0 and 1**.

- ACK numbers indicate the **next expected frame number**.

This simple numbering ensures that:

- Duplicate frames are detected.
- Only new frames are accepted and delivered.

Figure 11.12 *Flow diagram for Example 11.3*



2.6.3 Piggybacking

The protocols discussed so far assume **one-way data transmission**.

In **two-way communication**, both nodes send data and acknowledgments.
To improve efficiency, **piggybacking** is used.

Piggybacking:

- The ACK is **attached to a data frame** traveling in the opposite direction.
- This reduces the number of separate ACK frames.

Although piggybacking improves efficiency, it makes the protocol **more complex** and is therefore less commonly used at the data-link layer. It is discussed in detail in advanced protocol studies.

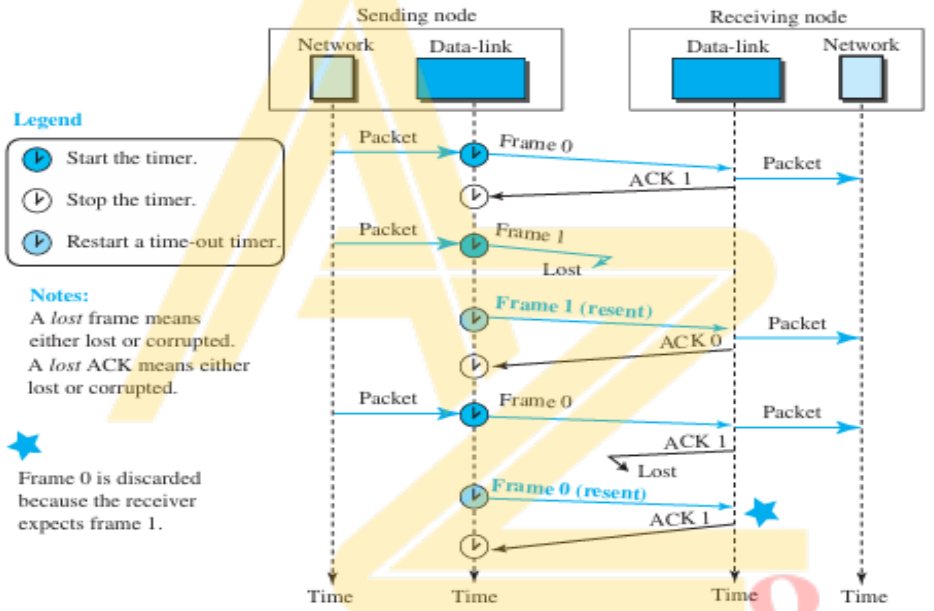
2.7 HDLC (High-Level Data Link Control)

High-Level Data Link Control (HDLC) is a **bit-oriented data-link layer protocol** used for communication over **point-to-point** and **multipoint** links. It is based on the **Stop-and-Wait** and sliding window concepts discussed earlier.

Although HDLC itself is not widely used today in its original form, it is **very important conceptually** because many modern protocols such as **PPP**, **Ethernet**, and **wireless LAN protocols** are derived from HDLC principles.

HDLC provides **reliable communication** by supporting **framing, flow control, error control, and connection management**.

Figure 11.13 *Flow diagram for Example 11.4*



2.7.1 Configurations and Transfer Modes

HDLC supports different configurations and transfer modes to suit various network requirements.

Normal Response Mode (NRM)

In **Normal Response Mode (NRM)**, the configuration is **unbalanced**.

- There is **one primary station** and **one or more secondary stations**.
- The **primary station controls communication**.

- A secondary station **can send data only when requested** by the primary station.

NRM can be used in both **point-to-point** and **multipoint** networks. This mode is suitable for systems where one device must control others, such as older mainframe-based networks.

Asynchronous Balanced Mode (ABM)

In **Asynchronous Balanced Mode (ABM)**, the configuration is **balanced**.

- The link is **point-to-point**.
- Both stations act as **peers**.
- Either station can send commands or responses at any time.

ABM is the **most commonly used mode today** because it allows full two-way communication without a master–slave relationship.

Figure 11.14 *Normal response mode*

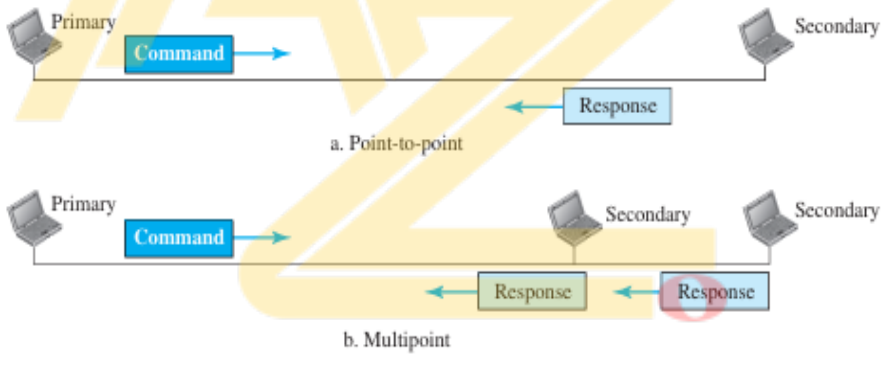


Figure 11.15 *Asynchronous balanced mode*



2.7.2 Framing in HDLC

To support different configurations and functions, HDLC defines **three types of frames**:

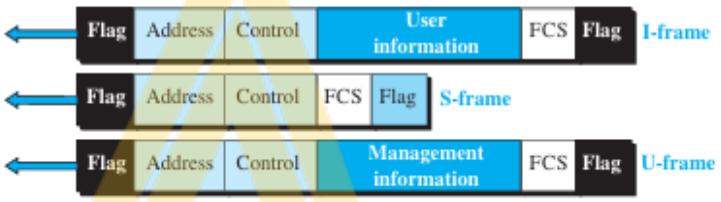
1. **Information frames (I-frames)**
2. **Supervisory frames (S-frames)**
3. **Unnumbered frames (U-frames)**

Each frame acts as an **envelope** that carries a specific type of information.

HDLC Frame Structure

An HDLC frame can contain **up to six fields**:

Figure 11.16 *HDLC frames*



1. **Flag field**
2. **Address field**
3. **Control field**
4. **Information field**
5. **Frame Check Sequence (FCS)**
6. **Flag field**

In continuous transmission, the **ending flag of one frame can act as the starting flag of the next frame**.

Flag Field

The flag field contains a **unique bit pattern (01111110)**.

- It marks the **beginning and end of a frame**.

- HDLC uses **bit stuffing** to ensure this pattern does not appear in the data accidentally.

Address Field

The address field identifies the **secondary station**.

- If the frame is sent by a **primary station**, the address specifies the **destination**.
- If sent by a **secondary station**, it specifies the **source**.
- The address field can be **one or multiple bytes long**, depending on the network.

Control Field

The control field is **one or two bytes long** and is responsible for:

- Identifying the **frame type**
- Providing **flow control**
- Providing **error control**

The format of this field depends on whether the frame is an **I-frame**, **S-frame**, or **U-frame**.

Information Field

- In **I-frames**, it carries **user data** from the network layer.
- In **U-frames**, it carries **management and control information**.
- **S-frames do not contain an information field**.

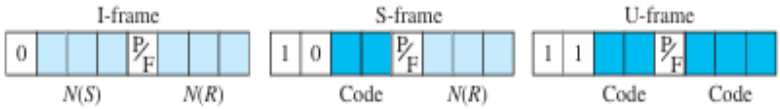
FCS (Frame Check Sequence)

The FCS field contains a **CRC** for error detection.

- It can be **2 bytes or 4 bytes long**.
- It allows the receiver to detect corrupted frames.

Control Field Formats

Figure 11.17 *Control field format for the different frame types*



Control Field for I-Frames

I-frames are used to **carry user data** and may also include **piggybacked acknowledgments**.

- The **first bit is 0**, indicating an I-frame.
- **N(S)** (3 bits): sequence number of the frame.
- **N(R)** (3 bits): acknowledgment number.
- **P/F bit**: Poll/Final bit used for control purposes.

The **P/F bit** means:

- **Poll** when sent by a primary station.
- **Final** when sent by a secondary station.

Control Field for S-Frames

S-frames are used **only for flow and error control**.

- The **first two bits are 10**, indicating an S-frame.
- They contain **no user data**.
- The **code field (2 bits)** defines the S-frame type.
- **N(R)** gives the acknowledgment or negative acknowledgment number.

Types of S-frames:

- **RR (Receive Ready)**: acknowledges frames and indicates readiness.
- **RNR (Receive Not Ready)**: acknowledges frames but indicates the receiver is busy.
- **REJ (Reject)**: requests retransmission using Go-Back-N.
- **SREJ (Selective Reject)**: requests retransmission of a specific frame.

Control Field for U-Frames

U-frames are used for **connection management and control**.

- They can carry **management information**.
- They do **not use sequence numbers**.
- The control field contains **codes before and after the P/F bit**, allowing up to **32 different U-frame types**.

Examples of U-frames include:

- **SABM** (Set Asynchronous Balanced Mode)
- **UA** (Unnumbered Acknowledgment)
- **DISC** (Disconnect)

Connection Management Using U-Frames

HDLC uses U-frames to **establish and release connections**.

- The sender requests a connection using **SABM**.
- The receiver confirms using **UA**.
- Data transfer then takes place.
- The connection is terminated using **DISC**, confirmed again by **UA**.

Piggybacking in HDLC

HDLC supports **piggybacking**, where:

- Acknowledgment information is **combined with data frames**.
- This improves efficiency by reducing the number of separate ACK frames.

Piggybacking is used mainly with **I-frames** and is effective in **two-way communication**.

2.8 Point-to-Point Protocol (PPP)

The **Point-to-Point Protocol (PPP)** is one of the most widely used **data-link layer protocols** for **point-to-point communication**. It is commonly used by Internet users to connect their personal computers to an **Internet Service Provider (ISP)**, especially over **dial-up telephone lines** using modems.

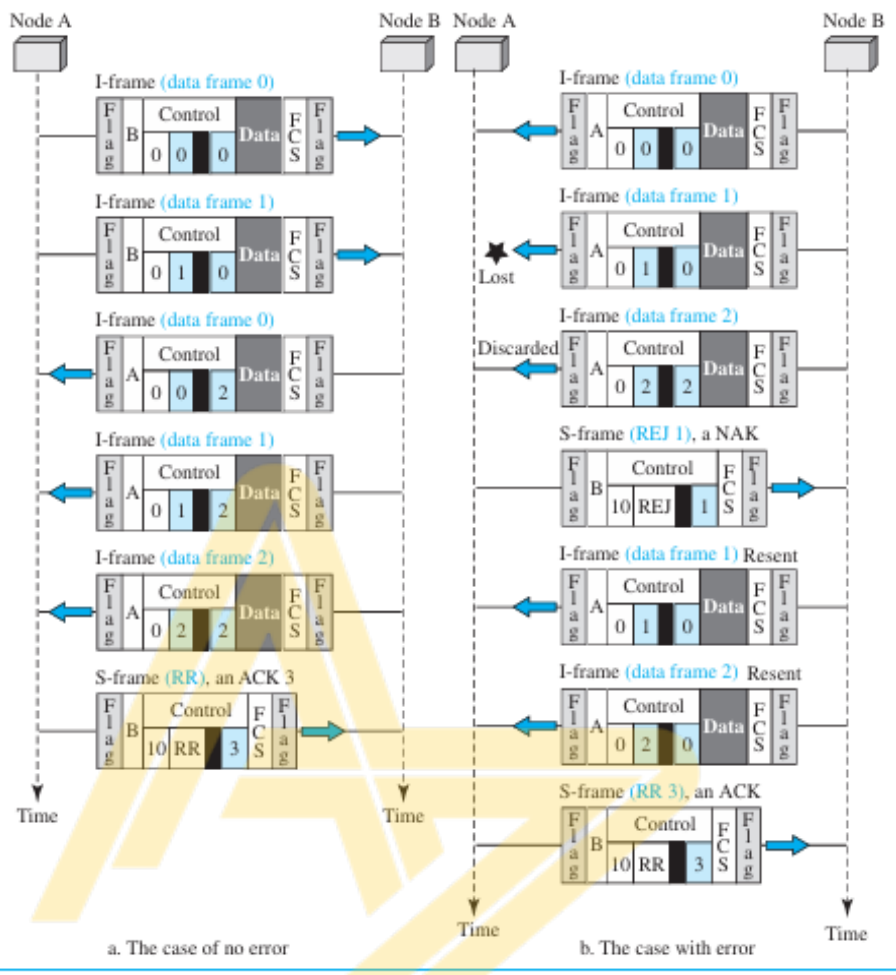
While the **physical layer** provides the actual transmission of bits over the telephone line, **PPP operates at the data-link layer** to control, manage, and organize the transfer of data. Because of its simplicity, flexibility, and wide compatibility, PPP has become the **most popular point-to-point protocol**.

2.8.1 Services of PPP

PPP is designed to provide only the **essential services** needed for point-to-point communication, while deliberately omitting some complex features to keep the protocol simple.

Services Provided by PPP

Figure 11.19 *Example of piggybacking with and without error*



PPP provides the following important services:

- PPP defines a **standard frame format** for communication between two devices.
- It supports **link establishment, configuration, and termination** through negotiation between the two endpoints.
- PPP can carry data from **multiple network-layer protocols**, not just IP.
- It provides **optional authentication**, allowing verification of user identity.
- PPP supports **network address configuration**, which is useful for assigning temporary IP addresses to users.

- An extended version called **Multilink PPP** allows data transmission over **multiple physical links** simultaneously.

These features make PPP highly suitable for Internet access and remote connections.

Services Not Provided by PPP

To maintain simplicity, PPP intentionally **does not provide some traditional data-link services**:

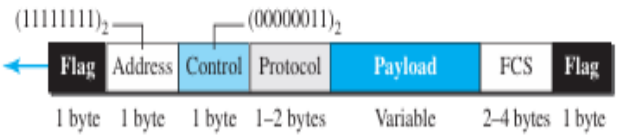
- PPP does **not provide flow control**, meaning the sender does not wait to check whether the receiver is overloaded.
- PPP provides only **error detection** using CRC; it does not perform error correction.
- Corrupted frames are **silently discarded**, and recovery is handled by higher-layer protocols.
- PPP does **not use sequence numbers**, so packets may arrive **out of order**.
- PPP does **not support multipoint addressing**, making it unsuitable for multipoint networks.

2.8.2 PPP Framing

PPP uses a **byte-oriented (character-oriented) framing technique**. Each PPP frame has a well-defined structure to ensure proper communication.

PPP Frame Fields

Figure 11.20 *PPP frame format*



A PPP frame consists of the following fields:

1. **Flag Field**
   The frame begins and ends with a 1-byte flag having the bit pattern **01111110**. This

flag marks the boundaries of a frame.

2. **Address Field**
   The address field is always set to **11111111**, indicating a broadcast address. Since PPP is point-to-point, this field has no real addressing function.

3. **Control Field**
   The control field is always set to **00000011**, similar to unnumbered frames in HDLC. This reflects the fact that PPP does not support flow control or sequencing.

4. **Protocol Field**
   This field identifies the type of data carried in the payload, such as IP data, LCP packets, authentication packets, or NCP packets.

5. **Payload Field**
   The payload carries user data or control information. Its default maximum size is **1500 bytes**, but this can be negotiated. Byte stuffing is used if special flag patterns appear in the data.

6. **FCS (Frame Check Sequence)**
   The FCS field contains a **CRC** (2 or 4 bytes) used for error detection.

Byte Stuffing in PPP

Since PPP is byte-oriented, **byte stuffing** is used to avoid confusion with flag bytes.

- The **escape byte (01111101)** is inserted before any flag-like byte appearing in the data.
- If the escape byte itself appears in the data, it is also escaped.
- At the receiver, the extra escape bytes are removed to recover the original data.

2.8.3 Transition Phases of PPP

A PPP connection passes through several **distinct phases**, represented using a finite state machine.

1. **Dead Phase**
   There is no physical connection, and the line is idle.

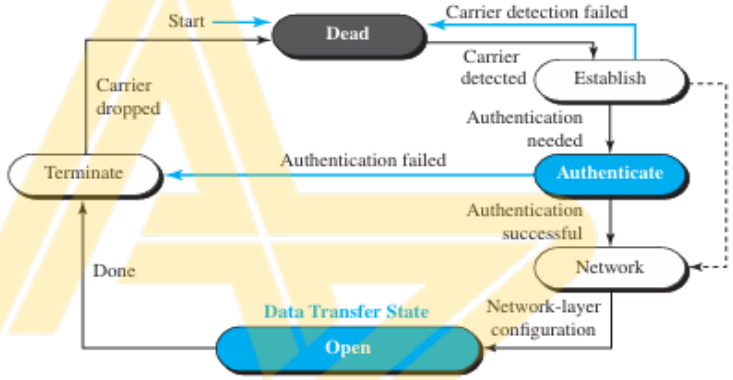
2. **Establish Phase**
   The physical connection is detected, and the **Link Control Protocol (LCP)** negotiates link options.

3. **Authenticate Phase (Optional)**
   If authentication is required, **PAP or CHAP** is used to verify user identity.

4. **Open (Data Transfer) Phase**
   Data transmission begins after successful link and network-layer configuration.

5. **Terminate Phase**
   One endpoint requests termination, and the connection is gracefully closed.

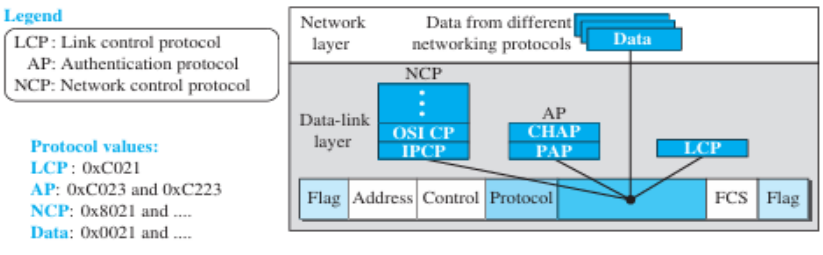
Figure 11.21 *Transition phases*



2.8.4 Multiplexing in PPP

PPP supports **multiple control and data protocols** using multiplexing. The protocol field in the PPP frame identifies which protocol is currently active.

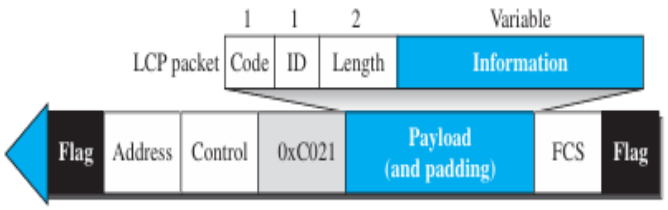
Figure 11.22 *Multiplexing in PPP*



PPP uses three major protocol categories:

Link Control Protocol (LCP)

Figure 11.23 *LCP packet encapsulated in a frame*



LCP is responsible for:

- Establishing the link
- Negotiating configuration options
- Monitoring link quality
- Terminating the link

All LCP packets are carried with the protocol field set to **0xC021**.
LCP packets include configuration requests, acknowledgments, termination messages, and diagnostic messages.

Table 11.1 *LCP packets*

Code	Packet Type	Description
0x01	Configure-request	Contains the list of proposed options and their values
0x02	Configure-ack	Accepts all options proposed
0x03	Configure-nak	Announces that some options are not acceptable
0x04	Configure-reject	Announces that some options are not recognized
0x05	Terminate-request	Request to shut down the line
0x06	Terminate-ack	Accept the shutdown request
0x07	Code-reject	Announces an unknown code
0x08	Protocol-reject	Announces an unknown protocol
0x09	Echo-request	A type of hello message to check if the other end is alive
0x0A	Echo-reply	The response to the echo-request message
0x0B	Discard-request	A request to discard the packet

Authentication Protocols

PPP supports two authentication protocols:

Password Authentication Protocol (PAP)

PAP is a **simple two-step authentication method**:

1. The user sends a **username and password**.
2. The system accepts or rejects the request.

PAP is simple but **not secure**, as the password is sent in plain text.

Challenge Handshake Authentication Protocol (CHAP)

CHAP is a **more secure three-step authentication method**:

1. The system sends a **challenge value**.
2. The user responds with a value generated using the challenge and password.
3. The system verifies the response and grants or denies access.

The password is **never transmitted**, making CHAP more secure than PAP.

Network Control Protocols (NCPs)

NCPs configure the link to carry **specific network-layer protocols**.

- Each network-layer protocol has its own NCP.
- NCPs do not carry data; they only configure the link.

IPCP (Internet Protocol Control Protocol)

IPCP configures the link for **IP packet transmission**.

- It assigns IP addresses.
- It negotiates IP-related options.
- IPCP packets use protocol value **0x8021**.

After IPCP completes, IP data packets are transmitted using protocol value **0x0021**.

Multilink PPP

Multilink PPP allows data to be transmitted over **multiple physical links**.

- A logical PPP frame is split into fragments.
- Each fragment is sent in a separate PPP frame.
- Sequence numbers are added to reassemble fragments correctly.

This improves bandwidth utilization but increases complexity.

2.9 RANDOM ACCESS

Random access methods, also called **contention methods**, are media access techniques in which **no station has priority over another** and **no centralized controller** decides who can transmit. Every station independently decides when to transmit based on the **current condition of the shared medium**.

In this approach, a station with data to send first checks whether the medium is **idle or busy**. If the medium is idle, the station may transmit according to the rules of the protocol. If the medium is busy, the station waits or follows a backoff procedure.

These methods are called **random access** because:

1. There is **no fixed schedule** for transmission.
2. Stations **compete randomly** for access to the medium.

Since multiple stations may attempt to transmit at the same time, **collisions can occur**, leading to corrupted frames. To handle this, random access protocols define rules that answer the following questions:

- When can a station transmit?
- What should it do if the channel is busy?
- How does it detect success or failure?

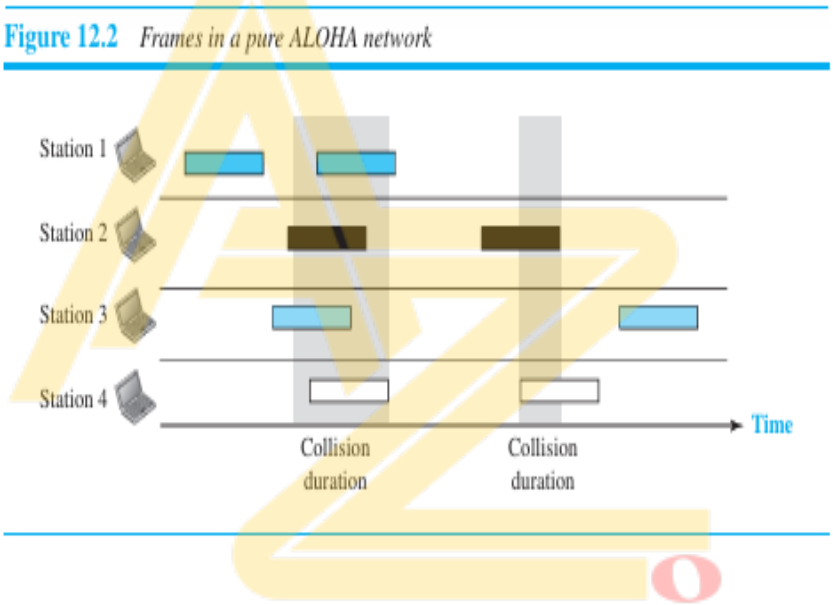
- What action should be taken after a collision?

Random access techniques evolved in stages, starting from **ALOHA**, then **CSMA**, and finally into **CSMA/CD** and **CSMA/CA**.

2.9.1 ALOHA

ALOHA was the first random access protocol, developed at the **University of Hawaii** for wireless communication. It allows stations to transmit whenever they have data, without checking the channel first.

Because all stations share the same medium, **collisions are unavoidable** when two or more stations transmit simultaneously.



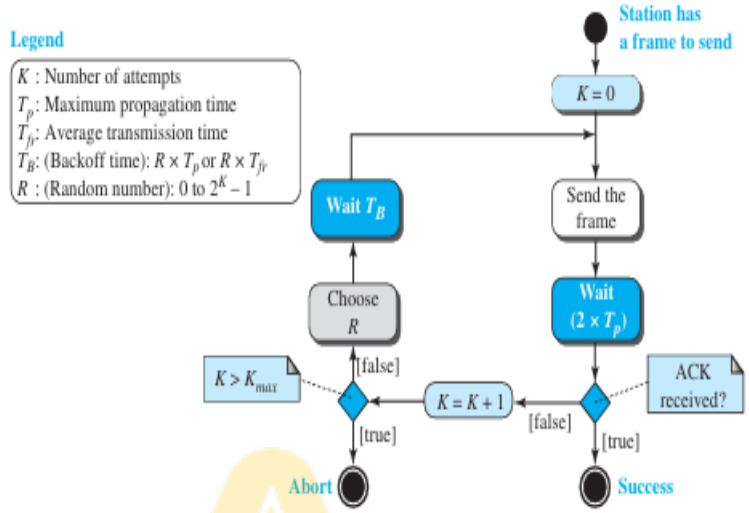
Pure ALOHA

In **Pure ALOHA**, a station sends a frame immediately whenever it has data to send. After transmission, it waits for an **acknowledgment (ACK)** from the receiver.

- If an ACK is received, the transmission is successful.
- If no ACK is received before a timeout, the station assumes a collision occurred and retransmits.

To avoid repeated collisions, each station waits for a **random backoff time** before retransmission. If a frame fails after a maximum number of attempts, it is discarded.

Figure 12.3 *Procedure for pure ALOHA protocol*



Vulnerable Time in Pure ALOHA

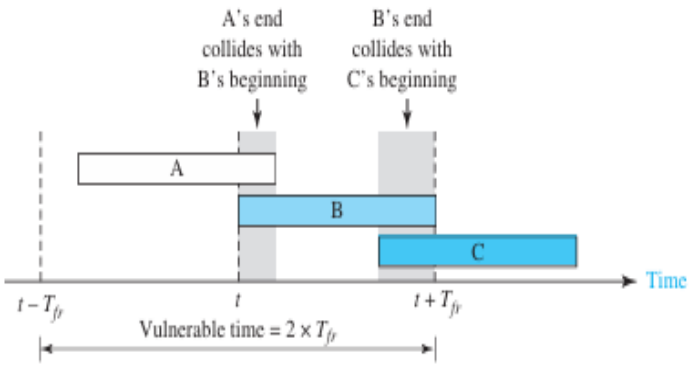
The **vulnerable time** is the period during which a frame is susceptible to collision.

This large vulnerable time makes Pure ALOHA inefficient.

Throughput of Pure ALOHA

The **maximum throughput** is only **18.4%**, which occurs when G=0.5G = 0.5G=0.5.
This means that less than one-fifth of transmitted frames are successful.

Figure 12.4 *Vulnerable time for pure ALOHA protocol*



Slotted ALOHA

Slotted ALOHA improves Pure ALOHA by dividing time into fixed **time slots**, each equal to one frame transmission time. Stations are allowed to transmit **only at the beginning of a slot**.

This synchronization reduces collisions and improves efficiency.

Vulnerable Time in Slotted ALOHA

The vulnerable time is reduced to:

$$\text{Vulnerable Time} = T_{fr}$$

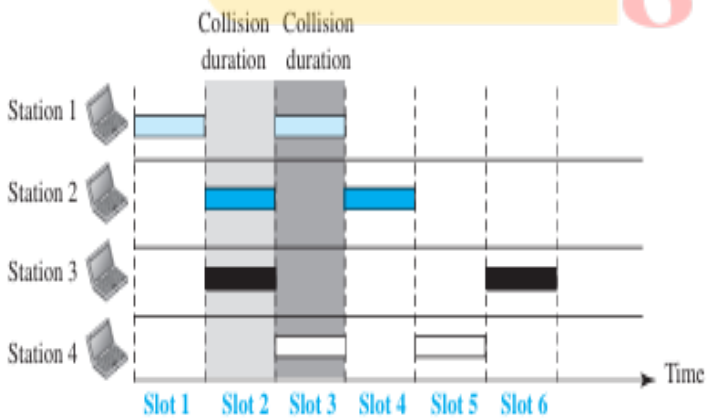
Throughput of Slotted ALOHA

The throughput is:

$$S = G \times e^{-G}$$

The **maximum throughput** is **36.8%**, which is double that of Pure ALOHA and occurs when $G = 1$.

Figure 12.5 *Frames in a slotted ALOHA network*



2.9.2 Carrier Sense Multiple Access (CSMA)

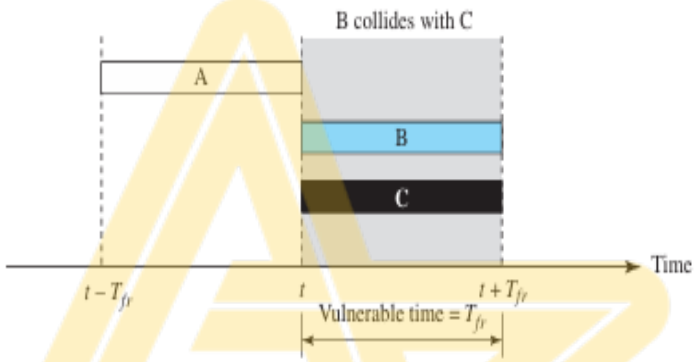
To further reduce collisions, **CSMA** requires a station to **sense the channel before transmitting**. This principle is known as **"listen before talk."**

Even though CSMA reduces collisions, it cannot eliminate them completely due to **propagation delay**. A station may sense the channel as idle while another station's signal is still propagating.

Vulnerable Time in CSMA

The vulnerable time in CSMA is equal to the **propagation delay (Tp)**, which is much smaller than in ALOHA.

Figure 12.6 *Vulnerable time for slotted ALOHA protocol*



Persistence Methods in CSMA

CSMA defines different strategies to decide **when to transmit** after sensing the channel.

1-Persistent CSMA

In this method, if the channel is idle, the station transmits immediately.
This method has a **high collision probability**, but low delay.

Non-Persistent CSMA

If the channel is busy, the station waits for a **random time** before sensing again.
This reduces collisions but may lead to **lower channel utilization**.

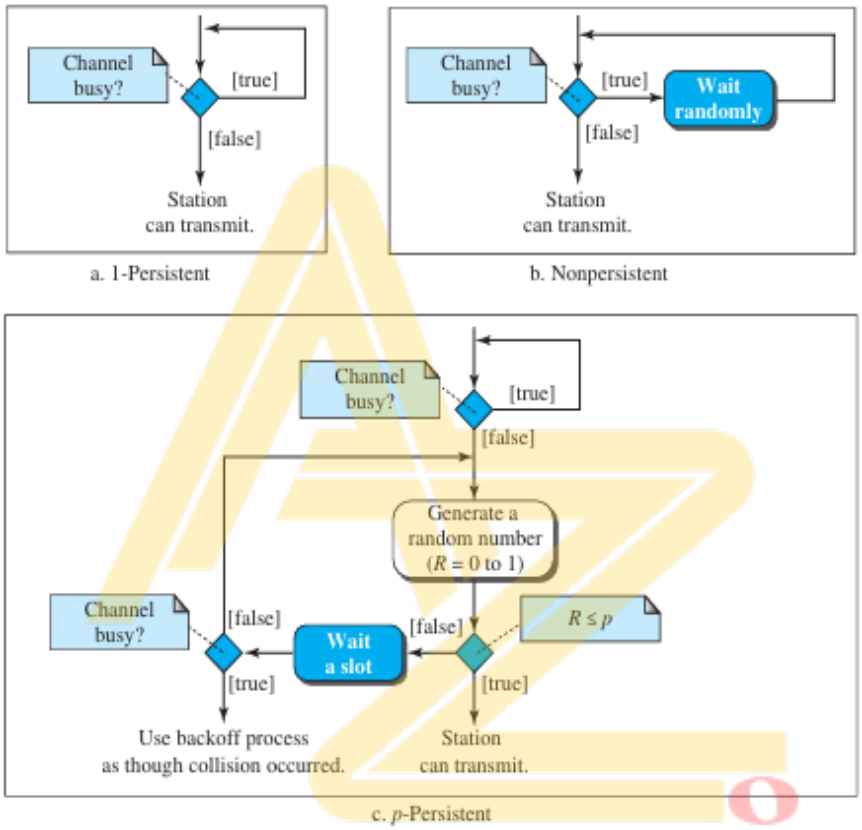
p-Persistent CSMA

This method is used in slotted channels.
When the channel is idle:

- The station transmits with probability **p**.
- With probability **(1 − p)**, it waits for the next slot.

This approach balances collision reduction and efficiency.

Figure 12.10 *Flow diagram for three persistence methods*

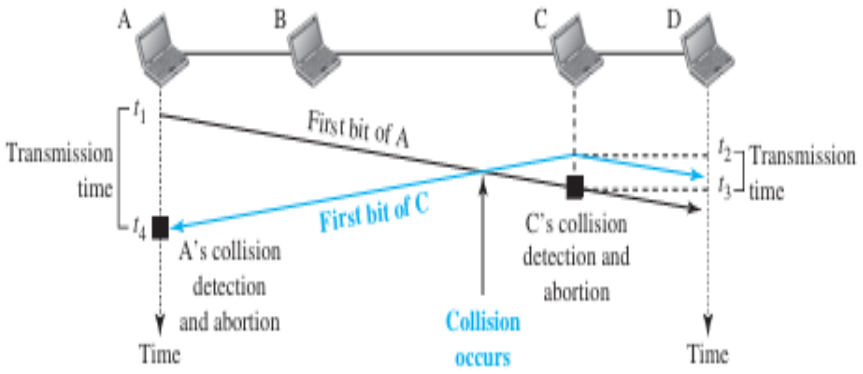


2.9.3 CSMA/CD (Collision Detection)

CSMA/CD enhances CSMA by allowing a station to **detect collisions while transmitting**. If a collision is detected:

- The station immediately stops transmission.
- It sends a **jamming signal** to alert other stations.
- It waits for a random backoff time and retries.

Figure 12.11 *Collision of the first bits in CSMA/CD*



Minimum Frame Size

For collision detection to work, the station must still be transmitting when a collision is detected. Therefore:

$$T_{fr} \geq 2 \times T_p$$

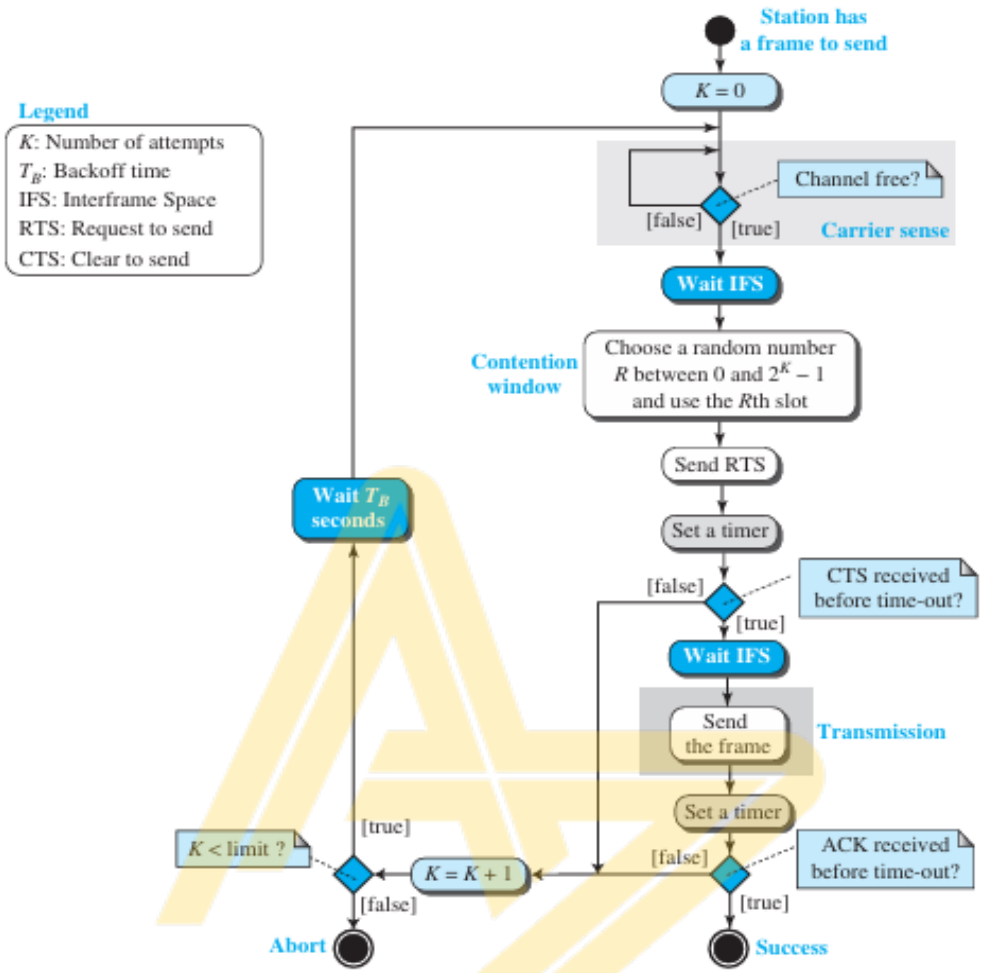
This requirement defines the **minimum frame size** (e.g., 64 bytes in Ethernet).

CSMA/CD was widely used in **traditional Ethernet (10 Mbps)** networks.

2.9.4 CSMA/CA (Collision Avoidance)

CSMA/CA is designed for **wireless networks**, where collision detection is difficult. Instead of detecting collisions, it **tries to avoid them**.

Figure 12.15 *Flow diagram of CSMA/CA*



CSMA/CA uses three main techniques:

Interframe Space (IFS)

Even if the channel is idle, a station waits for a short time before transmitting. This helps avoid collisions due to propagation delay.

Contention Window

Each station selects a **random waiting time** from a contention window before transmission. The window size increases after each failed attempt.

Acknowledgment

Every successfully received frame is acknowledged. If no ACK is received, the sender assumes failure and retransmits.

RTS/CTS and NAV

CSMA/CA uses **Request to Send (RTS)** and **Clear to Send (CTS)** frames to reserve the channel. Other stations hearing these frames set a **Network Allocation Vector (NAV)**, which tells them how long to remain silent.

This mechanism helps solve the **hidden station problem** in wireless networks.

2.10 CONTROLLED ACCESS

In **controlled access methods**, stations do not transmit data randomly. Instead, they **coordinate with one another** to determine **which station has the right to use the channel at a given time**. A station is **not allowed to send data unless it has received permission** according to the rules of the access method.

Because permission is explicitly controlled, **collisions are completely avoided** in controlled access techniques. These methods are mainly used in networks where **predictability, fairness, and reliability** are more important than speed.

There are **three main controlled-access methods**:

1. Reservation
2. Polling
3. Token Passing

2.10.1 Reservation

In the **reservation method**, a station must **reserve the channel in advance** before transmitting data. Time is divided into **fixed intervals**, and each interval starts with a **reservation frame**, followed by data transmission.

If there are **N stations** in the network, the reservation frame contains **N reservation mini slots**, one mini slot for each station.

- Each station owns a specific mini slot.
- If a station has data to send, it marks its mini slot.

- Stations that have marked their mini slots are allowed to send data in the same interval, **one after another**, without collision.

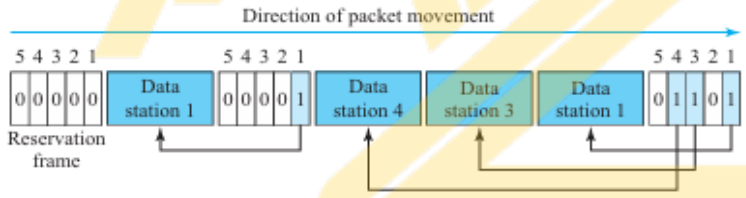
Only the stations that make a reservation are permitted to transmit, which ensures **collision-free communication**.

Key Characteristics of Reservation

- No collisions occur because access is pre-planned.
- Bandwidth is efficiently used when traffic is heavy.
- Overhead exists due to reservation frames, especially when few stations want to transmit.

Reservation access is suitable for **time-sensitive and predictable traffic**, such as real-time systems.

Figure 12.18 *Reservation access method*



2.10.2 Polling

Polling is a controlled-access method used in networks where one device is designated as the **primary station** and all other devices are **secondary stations**.

The **primary station controls the channel** and decides which secondary station can transmit. Secondary stations **cannot send data on their own**; they must wait until the primary station gives them permission.

There are **two main polling functions**:

Select Function

The **select function** is used when the **primary station wants to send data to a secondary station**.
Before sending data, the primary station sends a **Select (SEL) frame** to the intended secondary station. This frame alerts the secondary station and checks whether it is ready to receive data.
If the secondary station responds positively, the primary station proceeds with data transmission.

Poll Function

The **poll function** is used when the **primary station wants to receive data** from secondary stations.
The primary station **polls each secondary station one by one**, asking whether it has data to send.

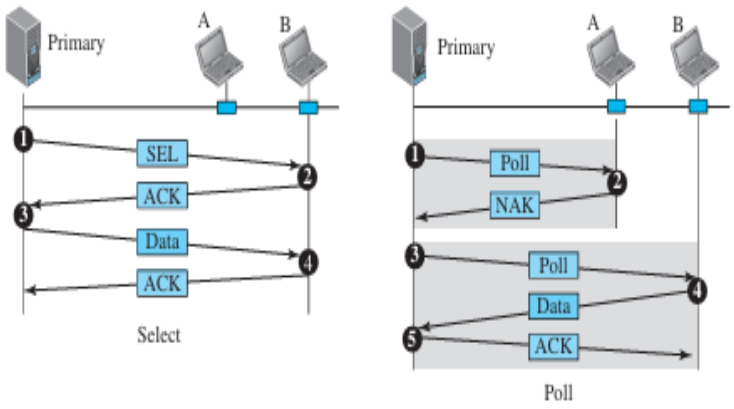
- If a secondary station has no data, it replies with a **NAK frame**.
- If it has data, it sends a **data frame**.
- After receiving the data, the primary sends an **ACK frame** to confirm successful reception.

This process continues until all secondary stations are checked.

Advantages and Disadvantages of Polling

- Collisions are avoided because only one station transmits at a time.
- The method is simple and well-organized.
- However, polling introduces **delay**, especially when many stations have no data.
- If the **primary station fails**, the entire network stops working.

Figure 12.19 *Select and poll functions in polling-access method*



2.10.3 Token Passing

In the **token-passing method**, stations are organized in a **logical ring**. Each station has a **predecessor** and a **successor**, even if the physical layout is not a ring.

A special control frame called a **token** circulates around the ring.

- **Only the station that holds the token is allowed to transmit data**.
- When a station receives the token:
    - If it has data to send, it holds the token and transmits.
    - If it has no data, it immediately passes the token to the next station.
- After finishing transmission, the station releases the token to its successor.

This method ensures **fair access**, because every station gets an equal chance to transmit.

Token Management

Token passing requires **token management** to ensure reliability:

- A station is allowed to hold the token only for a limited time.
- The system must detect and regenerate the token if it is lost.

- Priorities can be assigned so that high-priority stations can transmit earlier.
- Fault handling is required if a station holding the token fails.

Logical Ring in Token Passing

In token-passing networks, stations **do not need to be physically connected in a ring**. A **logical ring** can be formed using different physical topologies:

Physical Ring Topology

Stations are physically connected in a ring.

- Simple design.
- A single link failure can bring down the entire network.

Dual Ring Topology

A second ring runs in the opposite direction as a backup.

- Used in **FDDI and CDDI networks**.
- Provides high reliability.
- Requires extra hardware.

Bus Ring (Token Bus)

Stations are physically connected in a bus but form a logical ring.

- Each station knows the address of its successor.
- Used in **IEEE Token Bus LANs**.

Star Ring Topology

Stations are connected to a central hub that internally forms a ring.

- Easier to add or remove stations.

- More fault-tolerant.
- Used in **IBM Token Ring LANs**.

Figure 12.20 *Logical ring and physical topology in token-passing access method*

